Multiple Regression Code

Trong phần này mình sẽ:

- Viết Regression Model của mình, có sử dụng nhiều features.
- Viết lại các hàm Prediction, Gradient Descent để áp dụng được cho nhiều feature
- Sử dụng np.dot để vector hoá và cho sự tăng tốc và đơn giản hoá.

Các thư viện sẽ sử dụng:

- Numpy để tính toán (sử dụng array và tích vô hướng...)
- Matplotlib để vẽ biểu đồ minh hoạ

```
import copy, math
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('./deeplearning.mplstyle')
np.set_printoptions(precision=2)  # reduced display precision
on numpy arrays
```

Các ký hiệu sẽ sử dụng:

General Notation

General Notation	Description	Python (if applicable)
a	scalar, non-bold	
a	vector, bold	
A	matrix, bold capital	

Regression

General Notation	Description	Python (if applicable)
X	training example matrix	`X_train`
y	training example targets	`y_train`
x^(i), y^(i)	i-th Training Example	`X[i]`, `y[i]`
m	number of training examples	`m`
n	number of features in each example	`n`
w	parameter: weight	`w`
b	parameter: bias	`b`
f_{w,b}(x^(i))	The result model evaluation at x^(i) parameterized by w, b; f_{w,b}(x^(i)) = w · x^(i) + b	

- Trong phần này mình vẫn sẽ sử dụng ví dụ về việc dự đoán giá nhà, nhưng giờ sẽ có thể các feature khác. (Bao gồm kích thước, số phòng ngủ, số tầng và tuổi).

Size (sqft)	Number of Bedrooms	Number of floors	Age of Home	Price (1000s dollars)
2104	5	1	45	460
1416	3	2	40	232
852	2	1	35	178

- Code để cho vào 2 biến X và y, X_train là một matrix với shape (3,4)

```
X_train = np.array([[2104, 5, 1, 45], [1416, 3, 2, 40], [852,
2, 1, 35]])
y_train = np.array([460, 232, 178])
```

Với một dạng dữ liệu lớn hơn, công thức tổng quát cho vector X để huấn luyện có dạng: (Lưu ý số feature của mỗi training example bắt đầu từ số 0)

$$\mathbf{X} = \begin{pmatrix} x_0^{(0)} & x_1^{(0)} & \cdots & x_{n-1}^{(0)} \\ x_0^{(1)} & x_1^{(1)} & \cdots & x_{n-1}^{(1)} \\ \cdots & & & \\ x_0^{(m-1)} & x_1^{(m-1)} & \cdots & x_{n-1}^{(m-1)} \end{pmatrix}$$

Đối với các parameters w và b:

- W là một vector chứa các trọng số với n phần tử (tương đương với số feature)
- Trong dataset của mình thì n = 4
- Mình sẽ biểu diễn vector W dưới dạng cột với công thức chung (bắt đầu từ 0)

$$\mathbf{w} = \begin{pmatrix} w_0 \\ w_1 \\ \cdots \\ w_{n-1} \end{pmatrix}$$

- Để minh hoạ thì mình sẽ cho w và b vào các biến b_init, w_init tương ứng, hãy nhớ w là một vector 1-D:

```
b_init = 785.1811367994083
w_init = np.array([ 0.39133535, 18.75376741, -53.36032453, -2
6.42131618])
print(f"w_init shape: {w_init.shape}, b_init type: {type(b_in
it)}")
```

Nhắc lại một số lý thuyết về một Model chứa nhiều biến, ta có công thức chung:

$$f_{\mathbf{w},b}(\mathbf{x}) = w_0x_0 + w_1x_1 + ... + w_{n-1}x_{n-1} + b \tag{1}$$

Để rút ngắn lại mình sẽ biểu diễn dưới dạng tích vô hướng của 2 vector W và B:

$$f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b \tag{2}$$

Hàm prediction (giờ sử dụng tích vô hướng 2 vector w và x)

```
def predict(x, w, b):
    p = np.dot(x, w) + b
    return p
```

- x có shape (n,), là một example với nhiều feature

- w có shape (n,), vector cho trọng số
- b (scalar), bi
- p (scalar): prediction

Ta có phương trình tính Cost cho nhiều biến J(w,b) là:

$$J(\mathbf{w},b) = \frac{1}{2m} \sum_{i=0}^{m-1} (f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)})^2 \tag{3}$$

Ta có ứng dụng code:

```
def compute_cost(X, y, w, b):
    m = X.shape[0]
    cost = 0.0
    for i in range(m):
        f_wb_i = np.dot(X[i], w) + b           #(n,)(n,) = sc
alar (see np.dot)
        cost = cost + (f_wb_i - y[i])**2       #scalar
    cost = cost / (2 * m)                      #scalar
    return cost
```

- Mình cũng có thể sử dụng hàm predict vừa viết bên trên nhưng giờ sẽ trực tiếp thành np.dot(X[i], w) + b thay vì gọi hàm liên tục.
- X là data có shape (m,n) - m example, n features
- y là vector (n,) của các targets
- w và b là vector, scalar tương ứng cho các tham số
- Trả về cost (scalar

→ Hàm sẽ lấy m là số training example mình có, lặp trong vòng lặp qua từng example, sau đó sẽ tính prediction, và cộng tổng với hiệu bình phương của cost nói chung. Sau khi kết thúc vòng lặp thì sẽ chia cho 2*m như trong công thức.

Gradient Descent với nhiều biến.

- Ta nhắc lại vòng lặp Gradient Descent nhiều biến lặp đi lặp lại cho tới khi nào converge:

$$
\begin{aligned}
&\text{repeat until convergence: } \{ \\
&\qquad w_j = w_j - \alpha \frac{\partial J(\mathbf{w},b)}{\partial w_j} \quad \text{for j} = 0..\text{n-1} \\
&\qquad b \ \ = b - \alpha \frac{\partial J(\mathbf{w},b)}{\partial b} \\
&\qquad \}
\end{aligned} \tag{5}
$$

→ Nhớ rằng mỗi một trọng số của một ví dụ đều phải chạy vòng lặp này, do vậy ta có j = 0...n-1

- Đây là phần viết cụ thể của đạo hàm, theo từng trọng số và bias.

$$
\frac{\partial J(\mathbf{w},b)}{\partial w_j} = \frac{1}{m} \sum_{i=0}^{m-1} (f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)} \tag{6}
$$

$$
\frac{\partial J(\mathbf{w},b)}{\partial b} = \frac{1}{m} \sum_{i=0}^{m-1} (f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)}) \tag{7}
$$

Ta có function tính đạo hàm để ra Gradient (KHÔNG PHẢI GRADIENT DESCENT):

```
def compute_gradient(X, y, w, b):
    """
    Computes the gradient for linear regression
    Args:
      X (ndarray (m,n)): Data, m examples with n features
      y (ndarray (m,)) : target values
      w (ndarray (n,)) : model parameters
      b (scalar)       : model parameter

    Returns:
      dj_dw (ndarray (n,)): The gradient of the cost w.r.t. t
he parameters w.
```

```
      dj_db (scalar):       The gradient of the cost w.r.t. t
he parameter b.
    """
    m,n = X.shape           #(number of examples, number of f
eatures)
    dj_dw = np.zeros((n,))
    dj_db = 0.

    for i in range(m):
        err = (np.dot(X[i], w) + b) - y[i]
        for j in range(n):
            dj_dw[j] = dj_dw[j] + err * X[i, j]
        dj_db = dj_db + err
    dj_dw = dj_dw / m
    dj_db = dj_db / m

    return dj_db, dj_dw
```

Giải thích:

- Phần tính đạo hàm theo b có thể tính trực tiếp và cộng tổng vào sau mỗi lần lặp
- Còn vòng lặp bên trong là tính tất cả đạo hàm của các trong số từ w0 đến wj-1, sau đó mới cộng vào
- Sau khi đã có cả 2 tổng đạo hàm thì mới chia cho số training example (m) rồi trả về.

Hàm gradient descent hoàn chỉnh (có sử dụng các hàm vừa viết)

```
def gradient_descent(X, y, w_in, b_in, cost_function, gradien
t_function, alpha, num_iters):
    """
    Performs batch gradient descent to learn theta. Updates t
heta by taking
    num_iters gradient steps with learning rate alpha

    Args:
```

```
      X (ndarray (m,n))   : Data, m examples with n features
      y (ndarray (m,))    : target values
      w_in (ndarray (n,)) : initial model parameters
      b_in (scalar)       : initial model parameter
      cost_function       : function to compute cost
      gradient_function   : function to compute the gradient
      alpha (float)       : Learning rate
      num_iters (int)     : number of iterations to run gradi
ent descent

    Returns:
      w (ndarray (n,)) : Updated values of parameters
      b (scalar)       : Updated value of parameter
      """

    J_history = []
    w = copy.deepcopy(w_in)
    b = b_in

    for i in range(num_iters):
        dj_db,dj_dw = gradient_function(X, y, w, b)
        w = w - alpha * dj_dw
        b = b - alpha * dj_db

        if i<100000:
            J_history.append( cost_function(X, y, w, b))

        # Print cost every at intervals 10 times or as many i
terations if < 10
        if i% math.ceil(num_iters / 10) == 0:
            print(f"Iteration {i:4d}: Cost {J_history[-1]:8.2
f}   ")

    return w, b, J_history #return final w,b and J history fo
r graphing
```

Giải thích:

- Khai báo J_history là biến array để lưu thay đổi, tí nữa mình sẽ sử dụng để in ra và vẽ hình.
- sử dụng deep copy để tránh làm thay đổi w_in bên ngoài hàm.
- num_iters là số lần vòng lặp cho gradient descent (một tham số đầu vào)
- tính ra gradient bằng cách gọi hàm gradient_function bên trên
- Cập nhật cả w và b sau một vòng lặp, sau khi đã nhân với learning rate alpha
- Lưu vào J_history các sự thay đổi của cost sau từng vòng, i<100000 là để cho không lưu quá nhiều
- In ra màn hình khoảng 10 dòng trong suốt quá trình chạy Cost và số vòng lặp của Cost đấy
- Cuối cùng trả w và b đã tối ưu, và J_history

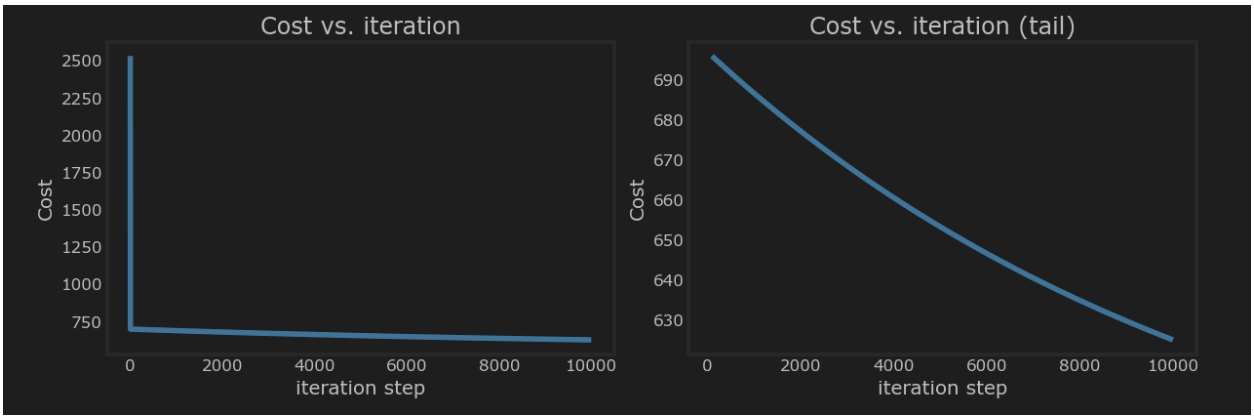
Phần vẽ hình minh hoạ:

```
# plot cost versus iteration
fig, (ax1, ax2) = plt.subplots(1, 2, constrained_layout=True,
figsize=(12, 4))
ax1.plot(J_hist)
ax2.plot(100 + np.arange(len(J_hist[100:])), J_hist[100:])
ax1.set_title("Cost vs. iteration");  ax2.set_title("Cost vs.
iteration (tail)")
ax1.set_ylabel('Cost')             ;  ax2.set_ylabel('Cost')
ax1.set_xlabel('iteration step')   ;  ax2.set_xlabel('iterati
on step')
plt.show()
```

- Hàm subplot(1,2) tạo ra một hàng và 2 cột, kiểu sẽ có 2 hình xếp ngang nhau
- constrained_layout = true để hình tự căn lề, không bị mất chữ
- figsize chỉ định kích thước hình theo inch

- J_hist là array theo từng vòng lặp được lưu lại ở bên trên
- ax1.plot(J_hist) → vẽ hình theo đường thẳng cost đang giảm dần
- 100 + np.arange(len(J_hist[100:])
    - lấy số số phần tử còn lại từ vòng lặp số 100 trở đi len(J_hist[100:]
    - xong rồi tạo một numpy array bằng .arang(0, 1, 2...) với số số phần tử ấy, và +100 thành 101, 102

→ Trục x của biểu đồ ax2 sẽ khớp đúng vòng lặp bắt đầu từ 100, và trục y là cost từ 100 trở đi (J_hist[100:])

→ Chắc là do sau 100 vòng lặp đầu tiên thì sự thay đổi của cost sẽ khó quan sát hơn

- Phần còn lại là đặt tiêu đề, đặt tên các trục và cuối cùng hiển thị